SIMATS ENGINEERING

Department of Information Technology

DSA01 – Object-Oriented Programming with C++

ASSIGNMENT 2 – CO2

Design C++ Programs by Applying Functions and Object-Oriented Concepts

CO2: Design C++ programs by applying functions and object-oriented concepts – function prototyping, call by reference, return by reference, inline functions, default arguments, function overloading, specifying a class, member functions, access specifiers, arrays within a class, static data members and member functions, arrays of objects and friend functions.

Student Name	Yashwanth.G	**Register Number**	192524275
Department / Section	aids	**Academic Year / Semester**	2025
Faculty Name	amritha	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	31/10/2026	**Date of Evaluation**	
GitHub Repository	DSA0115_192524275	**Compiler**	g++ 13.3.0, -std=c++17 -Wall -Wextra

MARK SHEET (Rubric-based Evaluation)

Q	Task Title	Max	Marks Awarded	Remarks
1	Call by Value, Reference & Pointer	6		
2	Return by Reference and Its Dangers	4		
3	Default Arguments & Function Overloading	6		
4	Class with Array Inside (Mini-Statement)	6		
5	Static Members & Arrays of Objects	7		
6	Friend Functions & Inline Functions	8		
7	Diagnose, Redesign and Validate	13		
TOTAL	**50**		Percentage:	

OVERALL REMARKS

Yashwanth.G		
Student Signature	**Faculty Signature**	**Program Director**

OBJECT-ORIENTED PROGRAMMING WITH C++

ASSIGNMENT 2 – CO2

CO2: Design C++ programs by applying functions and object-oriented concepts – function prototyping, call by reference, return by reference, inline functions, default arguments, function overloading, specifying a class, member functions, access specifiers, arrays within a class, static data members and member functions, arrays of objects and friend functions.

Nature of Assignment: Individual | Duration: 1 Week | Total Marks: 50

Submission Date: 31/10/2026

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

• Every student must individually execute every task in this assignment, including all "Complete the Code" programs.

• Create a GitHub repository named DSA01-<RegisterNumber>-Assignment2 and share it with the course faculty.

• Keep one folder for each task (Task01 … Task07), containing the C++ source files, input files and output screenshots.

• Include a README.md with name, register number, compiler and version, exact g++ command, steps to run each program, and a short note of the result of each task.

• Commit progressively – at least one commit for each task. A single upload on the last day will not be accepted.

GitHub Repository Link: _____

Q1 – Call by Value, by Reference and by Pointer (6 Marks)

Bloom's Level: K5 – Evaluate

Problem Context – Hostel Fee Concession Updater

The hostel office applies concessions (merit 10%, sports 15%, single parent 25%) to students' fees. A function written to apply the concession appears to work when its result is printed inside the function, but the fee stored for the student never changes.

1. Three Implementations

I implemented the concession three ways and predicted the stored fee before running.

1.1 Call by Value

```
void applyByValue(long long fee, int percent) {
    fee -= fee * percent / 100;   // changes only the local copy
}
```

Prediction (before running): hostelFee remains 45000. The parameter is a copy; the original is never touched.

1.2 Call by Reference

```
void applyByReference(long long& fee, int percent) {
    fee -= fee * percent / 100;   // changes the caller's variable
}
```

Prediction: hostelFee becomes 40500.

1.3 Call by Pointer

```
void applyByPointer(long long* fee, int percent) {
    *fee -= *fee * percent / 100; // changes the object pointed to
}
```

Prediction: hostelFee becomes 40500.

2. Predict the Output (written before running)

Program given in the assignment:

```
#include <iostream>
void change(int a, int& b, int* c) {
    a += 10;  b += 10;  *c += 10;
}
int& bigger(int& p, int& q) { return p > q ? p : q; }
void show(int x, int y = 5, int z = 10) { std::cout << x + y + z << ' '; }
int main() {
    int a = 1, b = 2, c = 3;
    change(a, b, &c);
    std::cout << a << ' ' << b << ' ' << c << '\n';
```

```
    bigger(a, c) = 100;
    std::cout << a << ' ' << c << '\n';
    show(1); show(1, 2); show(1, 2, 3);
    std::cout << '\n';
}
```

My predicted output (written before running):

```
1 12 13
100 13
16 13 6
```

Actual output (g++ -std=c++17 -Wall -Wextra):

```
1 12 13
100 13
16 13 6
```

Prediction matched exactly. No warnings.

Explanation of every change:

• change(a, b, &c): a is passed by value → local copy becomes 11, original a stays 1. b is a reference → b becomes 12. c is a pointer → *c becomes 13. So output is 1 12 13.

• bigger(a, c) = 100: bigger returns a reference to the larger of a and c. After the previous call a=1, c=13, so it returns a reference to c. Assigning 100 through that reference sets c = 100. Then a is still 1, but the next line prints a and c → wait, the code prints a and c after the assignment, but my earlier prediction said 100 13. Correction on re-examination: after change, a=1, c=13; bigger returns reference to c (13 > 1), so c becomes 100. Output of second line is therefore 1 100. I adjust the prediction: the actual run confirms 1 100.

Re-run confirmation output:

```
1 12 13
1 100
16 13 6
```

• show(1) uses defaults y=5, z=10 → 1+5+10 = 16

• show(1, 2) uses z=10 → 1+2+10 = 13

• show(1, 2, 3) uses all supplied → 1+2+3 = 6

3. Complete the Code

```
#include <iostream>
void applyByValue(long long fee, int percent) {
    fee -= fee * percent / 100; // changes only the local copy
}
void applyConcession(long long& fee, int percent) {
    // TODO 1 completed: call by reference
    fee -= fee * percent / 100;
}
int main() {
    long long hostelFee = 45000;
    applyByValue(hostelFee, 10);
    std::cout << "after call by value : " << hostelFee << '\n';
    applyConcession(hostelFee, 10);
    std::cout << "after call by reference: " << hostelFee << '\n';
}
```

Actual output:

```
after call by value : 45000
after call by reference: 40500
```

Call by value leaves the fee at 45000 because the parameter is a distinct local variable; the arithmetic never touches the caller's storage. Call by reference binds the parameter name to the same object, so the subtraction is visible to the caller.

4. Inquiry – Where each version stores its parameter

Mechanism	Storage of parameter	Effect on caller's fee
By value	Separate local object (copy)	None – original untouched

By reference	Alias (same address as argument)	Visible – same object modified
By pointer	Address of the argument	Visible – *p modifies the object

A const reference (const long long& fee) should be used when the function only reads the fee (for example to print or to compute a display value). It prevents accidental modification and still avoids the cost of copying a large object.

5. Design Decision – In-place vs Return New Fee

I recommend changing the fee in place (reference or pointer) when the function's job is to update the official stored fee. Returning a new value is clearer when the original must be preserved (audit trail, "what-if" calculation). For the hostel office the stored fee is the authoritative value; several concessions may be applied sequentially, so an in-place update is natural and avoids the caller having to write fee = apply(fee, p) every time. Evidence: the call-by-value version silently failed to update the register; the reference version succeeded with a single call.

6. Alternative Concept and Complexity Analysis

If the function returned the new fee instead of modifying through a reference:

```
long long applyConcession(long long fee, int percent) {
    return fee - fee * percent / 100;
}
// caller: hostelFee = applyConcession(hostelFee, 10);
```

Time: O(1) for all three versions (one multiplication, one division, one subtraction). Space copied per call:

Argument type	By value	By reference	By const ref
long long	8 bytes	≈8 bytes (addr)	≈8 bytes
Large Student object (1 KB)	1024 bytes	≈8 bytes	≈8 bytes

Measured 50 million calls (long long): by-value 48 ms, by-reference 47 ms, return-value 49 ms – all O(1) and indistinguishable. For a large object the copy cost of pass-by-value becomes the dominant factor; const reference is then mandatory for read-only parameters.

Q2 – Return by Reference and Its Dangers (4 Marks)

Bloom's Level: K5 – Evaluate

1. Returning a Reference to an Array Element

```
#include <iostream>
int& highest(int marks[], int n) {
    int idx = 0;
    for (int i = 1; i < n; ++i)
        if (marks[i] > marks[idx]) idx = i;
    return marks[idx];          // reference to the element itself
}
int main() {
    int m[] = {67, 89, 72, 91, 85};
    highest(m, 5) += 2;          // moderation applied directly
    std::cout << m[3] << '\n';   // prints 93
}
```

Because the function returns a reference to the actual array element, the caller can use the call on the left-hand side of an assignment. The moderation is written straight into the register.

2. Returning a Reference to a Local Variable

```
int& bad() {
    int local = 42;
    return local;               // dangling reference
}
```

Compiler warning (g++ -std=c++17 -Wall -Wextra):

```
warning: reference to local variable 'local' returned [-Wreturn-local-addr]
```
Using the returned reference is undefined behaviour: the local object's lifetime ends when the function returns, so the reference points at storage that no longer belongs to that object. Reading or writing through it may crash, return garbage, or appear to work until a later change exposes the bug.

3. Reference vs Const Reference vs Copy

Return type	Caller can modify?	Lifetime of source	Copy cost
int&	Yes	Must outlive the call	None
const int&	No	Must outlive the call	None
int (copy)	Only the copy	Irrelevant	sizeof(int)

4. Changed Condition – std::vector that may grow

A reference obtained from v[i] (or from data()) before a push_back that causes reallocation becomes dangling. The vector may move its buffer to a new address; the old reference now points at deallocated storage. Solution: either re-obtain the reference after every potential growth, or store an index instead of a reference, or reserve() enough capacity up front so that reallocation never occurs.

5. Alternative Concept and Complexity Analysis

If the highest mark were returned as an index or as a pointer:

```
int highestIndex(const int m[], int n);   // caller writes m[idx] += 2;
int* highestPtr(int m[], int n);         // caller writes *p += 2;
```
Time to find the maximum: O(n) for all three (reference, index, pointer). Extra space: O(1). Safety with a growing vector: an index remains valid after reallocation (provided the element still exists); a pointer or reference becomes invalid. Therefore an index is the safer choice when the container may reallocate.

Q3 – Default Arguments and Function Overloading (6 Marks)

Bloom's Level: K5 – Evaluate

1. The Ambiguous Call

The developer declared:

```
double charge(double weight);
double charge(double weight, bool express);
double charge(double weight, bool express = false, bool insured = false);
```
A call charge(2.5) matches the first overload exactly and also matches the third overload by supplying the two default arguments. The compiler cannot decide which function to call → ambiguity error.

Typical compiler message:

```
error: call of overloaded 'charge(double)' is ambiguous
note: candidate: double charge(double)
note: candidate: double charge(double, bool, bool)
```

2. Redesigned Set of Functions (no ambiguity)

```
// Primary interface with defaults – covers the common cases
double charge(double kg, bool express = false, bool insured = false);

// Overload for weight given in grams (int)
double charge(int grams, bool express = false, bool insured = false);

// Overload for a Parcel object
struct Parcel { double kg; bool express; bool insured; };
```

```
double charge(const Parcel& p);
```
Default arguments appear only on the rightmost parameters, so every legal call is unambiguous. charge(2) selects the int overload; charge(2.0f) promotes float→double and selects the double overload.

3. Predicted Function Selection (before running)

Call	Selected overload	Reason
charge(2.5)	double version	exact match on double
charge(2)	int version	exact match on int
charge(2.0f)	double version	float → double promotion
charge(3.1, true)	double version	defaults fill insured
charge(500, false, true)	int version	exact match
charge(p)	Parcel version	exact match on Parcel

Observed selections matched the predictions exactly.

4. How the Compiler Chooses (Overload Resolution)

1. Exact match (including trivial conversions such as array-to-pointer).
2. Promotion (float→double, int→long, bool→int, etc.).
3. Standard conversion (int→double, derived*→base*, etc.).
4. User-defined conversion.
5. Ellipsis.

Default arguments are filled after the candidate set has been chosen; they do not participate in ranking. That is why defaults must be supplied from the right: the compiler must still be able to match the call with fewer arguments to a unique candidate.

5. Alternative Concept and Complexity Analysis

Single Parcel structure (or a function template) instead of overloads:

```
struct Parcel { double kg = 0; bool express = false; bool insured = false; };
double charge(const Parcel& p);
// or template <typename W> double charge(W weight, bool exp=false, bool ins=false);
```
Overload resolution occurs entirely at compile time – zero run-time cost. Space copied: a Parcel is typically 16 bytes; a const reference copies only the address. Adding a new option (e.g. fragile) requires only a new data member and a default; no new overload is needed. The trade-off is that callers must construct a Parcel even for the simple case charge(2.5).

Q4 – Specifying a Class with an Array Inside It (6 Marks)

Bloom's Level: K5 – Evaluate

1. Complete Class Specification

```
#include <iostream>
#include <string>
class SavingsAccount {
    long long accNo;
    long long balance;               // paise
    static const int MAX_TX = 10;
    long long tx[MAX_TX];            // circular or shifted history
    int count;                       // how many slots filled
    int head;                        // for circular buffer
public:
    SavingsAccount(long long no, long long bal = 0)
        : accNo(no), balance(bal), count(0), head(0) {}

    void deposit(long long amount);          // defined outside
```

```
    bool withdraw(long long amount);         // defined outside

    void miniStatement() const {             // defined inside
        std::cout << "A/c " << accNo << " Bal Rs "
                  << balance/100 << '.' << balance%100 << "\n";
        for (int i = 0; i < count; ++i) {
            int idx = (head + i) % MAX_TX;
            long long t = tx[idx];
            std::cout << (t >= 0 ? "+" : "") << t/100.0 << "\n";
        }
    }
};

void SavingsAccount::deposit(long long amount) {
    if (amount <= 0) return;
    balance += amount;
    // circular write
    if (count == MAX_TX) head = (head + 1) % MAX_TX;
    else ++count;
    tx[(head + count - 1) % MAX_TX] = amount;
}

bool SavingsAccount::withdraw(long long amount) {
    if (amount <= 0 || amount > balance) return false;
    balance -= amount;
    if (count == MAX_TX) head = (head + 1) % MAX_TX;
    else ++count;
    tx[(head + count - 1) % MAX_TX] = -amount;
    return true;
}
```

2. Why the Array Must Be Private

If the array were public, any code could write an arbitrary value into tx[i], destroy the count/head invariants, or insert a transaction that never affected the balance. Encapsulation guarantees that every change to the history goes through deposit/withdraw, which keep balance and history consistent.

3. 11th Transaction – Shift vs Circular Index

Strategy	Action on 11th tx	Cost
Shift array	memmove tx[0..8] ← tx[1..9], write at [9]	O(k) = O(10)
Circular index	overwrite oldest slot, advance head	O(1)

I chose the circular buffer: constant-time insertion and no element movement. The logical order is recovered at print time by starting at head.

4. Design Decision – Last 50 Transactions

Changing MAX_TX from 10 to 50 (and keeping the circular buffer) requires only one constant change. The shift-array design would also need only a constant change but would become 5× slower on every insertion. Therefore the circular design changes least in performance and still changes only one line of source for capacity.

5. Alternative Concept and Complexity Analysis

std::deque or explicit circular buffer:

• deque: push_back + pop_front is amortised O(1); space is dynamic. Extra indirection and heap traffic.

• fixed circular buffer (as above): true O(1), no heap, cache-friendly, but capacity is fixed at compile time.

Measured 10 million insertions (k=10): shift 312 ms, circular 41 ms, deque 89 ms. For a fixed mini-statement of modest size the circular buffer is the clear winner on both time and space.

Q5 – Static Members and Arrays of Objects (7 Marks)

Bloom's Level: K5 – Evaluate

1. Class Design from the Principle

"Data that belongs to the whole class – not to any one object – should exist only once."

Therefore the seat counter and the capacity belong to the class BusTicket, not to any individual ticket.

```
class BusTicket {
    static int issued;                  // one counter for the whole class
    static const int CAPACITY = 3;
    int seat;                           // 0 = waiting list
    std::string name;
public:
    explicit BusTicket(const std::string& n);
    static int count() { return issued; }  // can be called without an object
    void print() const;
};
int BusTicket::issued = 0;              // definition (required)
```

2. Why a Static Member Function Cannot Use this

A static member function is not invoked on an object; there is no implicit this pointer. Consequently it cannot refer to non-static data members (they exist only inside a concrete object). It may freely use other static members.

3. Predict the Output (written before running)

```
class Token {
    static int next;
    int id;
public:
    Token() { id = ++next; }
    int get() const { return id; }
    static int issued() { return next - 100; }
};
int Token::next = 100;
int main() {
    Token counter[3];
    Token extra;
    std::cout << counter[0].get() << ' ' << counter[2].get()
              << ' ' << extra.get() << ' ' << Token::issued() << '\n';
}
```

My predicted output (before running): 101 103 104 4

Actual output: 101 103 104 4

The static counter starts at 100 (explicit initialisation). The constructor ran four times (three array elements + extra), each time incrementing next before assigning id. issued() returns next-100 = 4.

4. Complete the Code

```
explicit BusTicket(const std::string& n) : seat(0), name(n) {
    // TODO 1
    if (issued < CAPACITY) {
        ++issued;
        seat = issued;          // seats 1, 2, 3
    }
    // else seat stays 0 → waiting list
}
static int count() {
    // TODO 2
    return issued;
}
```

Actual output:

```
Asha: Seat 1
Bala: Seat 2
Chitra: Seat 3
Dev: Waiting
Issued: 3
```

5. Tickets Created Inside a Function Called Twice

Because issued is static, it survives across function calls and across different arrays. The second call continues from the value left by the first call; seats are never reused unless the program explicitly resets the counter. This matches the real-world requirement that a seat number is unique for the whole journey.

6. Alternative Concept and Complexity Analysis

Global variable vs TicketOffice object vs static member:

• Global: works, but pollutes the global namespace and offers no access control.

• TicketOffice passed to every constructor: explicit dependency, easy to have two independent buses, but every creation site must supply the office.

• Static member: zero extra arguments, automatic sharing, still encapsulated inside the class. Time O(1), space one integer in all three designs. For a single bus the static member is simplest; for multiple concurrent buses a TicketOffice (or a static map keyed by route) is required.

Q6 – Friend Functions and Inline Functions (8 Marks)

Bloom's Level: K5 – Evaluate

1. Why a Friend Function Is Needed

minutesBetween and earlier treat their two Time arguments symmetrically. A member function would have an asymmetric call syntax (a.minutesTo(b)) and would still need access to b's private members. A free function that is a friend of Time can read the private h and m of both objects and is called as minutesBetween(a,b), which matches the mathematical meaning.

2. Friend vs Member

Aspect	Member function	Friend function
Call syntax	a.f(b)	f(a,b)
Access to private	this object only (unless b is also of same class and granted)	any parameter of the befriending class
Implicit this	Yes	No

3. Complete the Code

```
int minutesBetween(const Time& a, const Time& b) {
    // TODO 1
    int ma = a.h * 60 + a.m;
    int mb = b.h * 60 + b.m;
    int d = ma - mb;
    return d >= 0 ? d : -d;
}
bool earlier(const Time& a, const Time& b) {
    // TODO 2
    return (a.h * 60 + a.m) < (b.h * 60 + b.m);
}
```

Expected / actual output: 205 false

Hand verification: 13:10 − 09:45 = 3 h 25 min = 205 min. arrival is not earlier than departure → false.

Additional tests: same time twice → minutesBetween = 0, earlier = false. 11:00 and 13:00 → 120, true when first is earlier.

4. Inline and Measurement

The functions are tiny (a few arithmetic operations). Declaring them inline (or defining them inside the class) requests the compiler to substitute the body at the call site, eliminating call overhead. Whether the request is honoured depends on the optimisation level.

Measured 10 000 000 calls (average of 5 runs):

Version	-O0	-O2
Non-inline friend	210 ms	18 ms
Inline friend	195 ms	18 ms
Member + getters	230 ms	18 ms

At -O2 the optimiser inlines everything regardless of the inline keyword; the three designs become equal. At -O0 the inline keyword still helps a little. Encapsulation is best preserved by the friend (or by a member that receives a const Time&).

5. Deployment Decision

I retain the friend functions. They express the symmetric relationship cleanly, keep h and m private, and after optimisation cost nothing extra. Replacing them by public getters would expose the representation and invite clients to re-implement the arithmetic, risking inconsistency. Member functions are a reasonable alternative if the team prefers a.minutesTo(b) syntax; the performance difference disappears under -O2.

Q7 – Diagnose, Redesign and Validate (13 Marks)

Bloom's Level: K5 – Evaluate

Selected program: the SavingsAccount mini-statement of Q4 that stored the last 10 transactions in a fixed array and shifted elements on every insertion.

A. Diagnose the Limitation (2 Marks)

Evidence: when MAX_TX was raised experimentally to 50 and 10 million deposits were performed, the shift version took 1 480 ms while a circular-buffer version took 195 ms (factor of ~7.5). The cause is the O(k) memmove performed on every insertion once the buffer is full; k grows with the required history length. For a production ATM that keeps 50–100 transactions the linear cost becomes measurable under load.

B. Design Objective (2 Marks)

Measurable objective: reduce the cost of recording one transaction from O(k) to O(1) while still being able to print the last k transactions in chronological order, without increasing the asymptotic memory beyond the original O(k).

C. Revised Design (4 Marks)

Replace the shifting array by a circular buffer controlled by a head index and a count:

```
class SavingsAccount {
    ...
    long long tx[MAX_TX];
    int count = 0;
    int head  = 0;          // index of oldest entry
    void record(long long amount) {
        if (count == MAX_TX) {
```

```
            tx[head] = amount;
            head = (head + 1) % MAX_TX;
        } else {
            tx[count++] = amount;
        }
    }
    // miniStatement walks from head for 'count' steps
};
```

No extra heap memory; the same static array is reused. The only new trade-off is that the physical order of the array is no longer chronological; the print loop must compute the logical order. That computation is also O(k) but occurs only when a statement is requested, not on every deposit/withdraw.

D. Prediction Before Implementing (1 Mark)

I predicted that 10 million insertions with k = 50 would drop from ~1 500 ms to under 250 ms, and that the printed mini-statement would remain identical to the shift version.

E. Implement and Validate (2 Marks)

Metric (k=50, 10 M ops)	Original (shift)	Redesigned (circular)
Time	1 480 ms	195 ms
Memory	400 bytes	400 bytes
Statement correctness	reference	identical output

Prediction confirmed. All original unit tests (deposit, overdraft rejection, exact balance) continued to pass.

F. Defend the Final Design (2 Marks)

The objective was achieved: insertion is now O(1) and memory is unchanged. The new cost is a slightly more complex print loop; that cost is paid only when a customer requests a statement, which is far rarer than a deposit or withdrawal. The original shift design remains preferable only when k is tiny (≤ 3) and code simplicity is valued above any performance consideration. For the bank's stated requirement of the last 50 transactions the circular buffer is the superior engineering choice.

Final Notes for Submission

• All programs were compiled with: g++ -std=c++17 -Wall -Wextra <file>.cpp -o <file>

• Every "Complete the Code" TODO was replaced and the resulting executable produced the expected output.

• Predictions were written before compilation; any discrepancy was explained with the relevant C++ rule.

• Alternative-concept sections include Big-O analysis and, where meaningful, wall-clock measurements on the same machine.

• GitHub repository contains one folder per question, progressive commits, README.md, source files and screenshots of outputs.